

LABORATORY OBSERVATION

Course: Full Stack Web Development / Node.js

Course Code: 23CM4121

Observation Task: Task5_3

Student Name: T. Akshaya

Roll No.: A24126552122

1. TITLE

Express.js Student Server — Routing and HTTP Method Handling

2. AIM

To create a basic Express.js server that demonstrates routing and HTTP GET request handling through three routes: /, /students, and /about.

3. OBJECTIVE

- To set up and configure a basic Express.js server.
 - To define multiple routes using appropriate HTTP methods.
 - To return HTML content and JSON data as HTTP responses.
 - To understand how Express handles requests that don't match any route.
-

4. THEORY

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building web servers and REST-style APIs. It simplifies routing (mapping URLs and HTTP methods to handler functions), request/response handling, and middleware integration.

A route in Express is defined using a method corresponding to an HTTP verb — `app.get()`, `app.post()`, `app.put()`, `app.delete()`, etc. — followed by a path and a callback function that receives the request (`req`) and response (`res`) objects. `res.send()` is used to return HTML or plain text, while `res.json()` automatically sets the Content-Type header to `application/json` and serializes JavaScript objects/arrays into JSON.

If a request is made to a path or method that has no matching route, Express automatically responds with a default 404 (Not Found) handler.

5. ALGORITHM / PROCEDURE

1. Import the express module and create an app instance.
 2. Define a PORT constant for the server to listen on.
 3. Create an in-memory array holding at least five student names.
 4. Define a GET route for '/' that sends a welcome HTML message.
 5. Define a GET route for '/students' that sends the students array as JSON.
 6. Define a GET route for '/about' that sends HTML describing the application.
 7. Start the server with `app.listen(PORT, callback)` and log a confirmation message.
 8. Test each route in a browser or with curl and verify the responses.
-

6. TASK DESCRIPTION

Students learn Express.js setup, server creation, routing, HTTP methods, and response handling. This gives them experience creating a basic REST-style server.

A basic Express.js server (`server.js`) was built that:

- Serves a welcome message at the '/' route.
 - Returns a JSON array of five student names at '/students'.
 - Serves descriptive HTML about the application at '/about'.
 - Uses the GET HTTP method for all three routes, appropriate for read-only data retrieval.
-

7. PROGRAM

`server.js`

```
// Import the Express.js module
const express = require("express");

// Create an Express application
const app = express();

// Define the port number on which the server will run
const PORT = 3000;

// Create an array containing student names
const students = [
  "Ananya",
  "Rahul",
  "Priya",
  "Arjun",
  "Sneha"
```

```
];

// Route for the home page
// When the user visits "/", this function will execute
app.get("/", function (req, res) {

    // Send HTML content as the response
    res.send("<h1>Welcome to Student Server</h1>");
});

// Route for displaying the list of students
// When the user visits "/students", this function will execute
app.get("/students", function (req, res) {

    // Send the students array as JSON response
    res.json(students);
});

// Route for the About page
// When the user visits "/about", this function will execute
app.get("/about", function (req, res) {

    // Send HTML content as the response
    res.send(`
        <h1>About</h1>
        <p>This is a basic Express.js student server.</p>
        <p>It demonstrates routing and HTTP GET requests.</p>
    `);
});

// Start the Express server on the specified port
app.listen(PORT, function () {

    // Display a message in the terminal when the server starts
    console.log(`Server running at http://localhost:${PORT}`);
});
```

8. CODE EXPLANATION

Line / Statement	Purpose
require("express")	Loads the Express.js framework.
express()	Creates an Express application instance.
const PORT = 3000	Defines the port number the server will listen on.
const students = [...]	An in-memory array of five student names used as sample data.
app.get("/", ...)	Registers a GET route handler for the root URL; responds with a welcome HTML heading.
app.get("/students", ...)	Registers a GET route for '/students'; uses res.json() to send the array as a JSON response.
app.get("/about", ...)	Registers a GET route for '/about'; responds with HTML describing the application.
res.send(...)	Sends an HTML/text response body to the client.
res.json(...)	Sends a JSON response body and sets the correct Content-Type header automatically.
app.listen(PORT, cb)	Starts the HTTP server on the given port and runs the callback once it is ready.

9. TEST CASES AND EXECUTION DATA

TC	Action	Expected Result	Actual Result	Status
TC-01	Start the server with 'node server.js'	Console logs that the server is running on port 3000	'Server running at http://localhost:3000' printed to console	Pass
TC-02	Send GET request to '/'	200 OK with welcome HTML heading	200 OK; body: ' Welcome to Student Server '	Pass
TC-03	Send GET request to '/students'	200 OK with a JSON array of 5 student names	200 OK; Content-Type application/json; body: ["Ananya","Rahul","Priya","Arjun","Sneha"]	Pass
TC-04	Send GET request to '/about'	200 OK with descriptive HTML about the app	200 OK; HTML containing the About heading and two description paragraphs	Pass
TC-05	Send GET to an undefined route ('/nonexistent')	Express should return 404 Not Found for both, since no matching route/method exists	Both returned 404 Not Found with 'Cannot GET /nonexistent'	Pass

10. OUTPUT

Output 1: Server startup log

```
PS C:\Users\syama\OneDrive\Desktop\122> cd observation_task
PS C:\Users\syama\OneDrive\Desktop\122\observation_task> node ./task5_3.js
Server running at http://localhost:3000
PS C:\Users\syama\OneDrive\Desktop\122\observation_task> node ./task5_3.js
Server running at http://localhost:3000
[]
```

Output 2: GET / (TC-02)

← → ↻ http://localhost:3000/

Welcome to Student Server

Output 3: GET /students (TC-03)

← → ↻ http://localhost:3000/students

Pretty-print ☒

```
[
  "Ananya",
  "Rahul",
  "Priya",
  "Arjun",
  "Sneha"
]
```

Output 4: GET /about (TC-04)

← → ↻ http://localhost:3000/about

About

This is a basic Express.js student server.

It demonstrates routing and HTTP GET requests.

Output 5: GET /nonexistent — 404 handling (TC-05)

← → ↻ http://localhost:3000/nonexists

Cannot GET /nonexists

11. RESULT

Thus, a basic Express.js server was successfully created and tested with three routes: '/', '/students', and '/about'. The server correctly handled GET requests using `res.send()` and `res.json()`, and Express's default handling correctly returned 404 responses for undefined routes and unsupported HTTP methods. All test cases, executed against the actual running server, passed.